

The historical divergence between HPC and cloud computing is narrowing. Traditionally, HPC systems have prioritized performance for tightly coupled scientific simulations on fixed resource allocations. Cloud platforms, on the other hand, were built to support resilience, elasticity, and changing resource availability, primarily for loosely coupled workloads such as web services.

Today, hardware innovation, the capital investment required for next-generation systems, and the growth of AI/ML workloads are bringing these environments closer together. HPC users increasingly consider cloud resources, while cloud providers offer specialized instances and high-performance networks to support their applications. This convergence presents an opportunity to broaden access to high-end computing, but it also exposes a fundamental question: how can applications designed for fixed resource allocations execute efficiently when the resources available to them change in number, performance, and availability?

The core problems are:

Handling resource interruptions: Conventional HPC execution commonly relies on checkpoint/restart to handle node loss. For resource revocations announced in advance, as in the case of preemptible spot instances, restarting the entire job can impose avoidable overhead. This cost is particularly significant for GPU-resident applications, where restarting also requires reconstructing device state and reinitializing the GPU execution environment.

Limited support for elasticity: Traditional HPC applications are commonly built to run on a fixed number of nodes. Shrinking or expanding a running application requires redistributing its work and preserving application state and communication across the change in membership. Without runtime support, these responsibilities fall on the application developer.

A rigid scheduler–runtime boundary: In conventional HPC deployments, the batch scheduler allocates a fixed set of nodes for a fixed duration, and the runtime treats that allocation as fixed for the lifetime of the job. The scheduler has limited information about whether an application could productively use more nodes or continue efficiently with fewer. The runtime, in turn, has limited ability to respond to changes in resource availability or cost. This separation restricts the decisions available to both layers, even when changing the allocation would benefit the application or the system.

In this thesis, we argue that efficient execution of HPC workloads on dynamic resources requires adaptivity in both the runtime system and the job scheduler. We develop a parallel runtime that supports efficient and portable overdecomposition on distributed GPU platforms, dynamic load balancing, and elastic execution. Overdecomposition separates application work from its placement on hardware; GPU load measurement and migration support allow the runtime to redistribute that work on heterogeneous resources as application load changes. Elastic execution extends this capability to changes in the allocation itself, allowing a running job to rescale while preserving state in surviving processes. When departing resources remain available long enough to evacuate their state, these changes require neither a job restart nor a disk-based or in-memory checkpoint.

We further investigate the co-design of this runtime with a scheduling layer that can revise allocations during execution. The scheduler and runtime exchange information about resource availability, application's parallel efficiency, and the cost of rescaling, enabling decisions that neither could make in isolation. Together, these contributions establish a foundation for parallel runtime systems that bring HPC execution efficiency to dynamic cloud environments, allowing changes in workload, hardware performance, and resource availability to be seamlessly accommodated during execution.